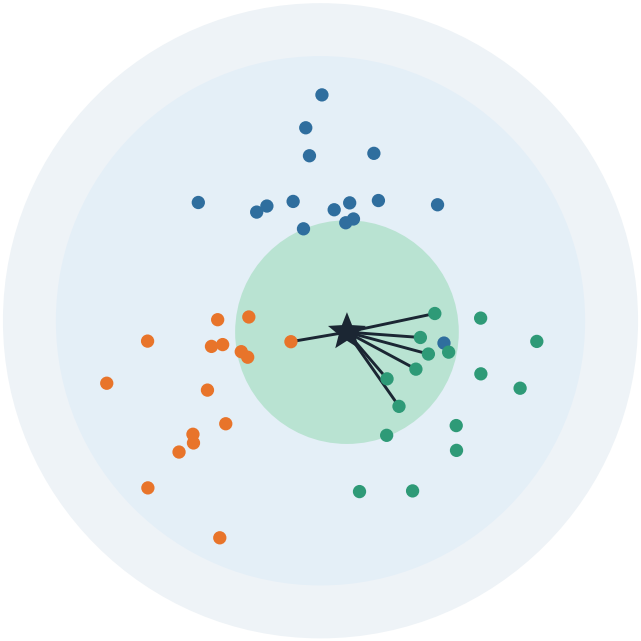
Ближайшие соседи

Урок



Первый из пяти уроков о моделях: ближайшие соседи, решающие деревья, линейная регрессия, логистическая регрессия и перцептрон, k-средних. Каждая модель — изнутри: что она считает, как учится и где ломается. Всё написано на numpy, а в последнем разделе урока то же делает sklearn, и видно, где его правила расходятся с точными. Этот урок начинается с того, что нужно во всех пяти: что такое модель и как мерить её качество. Ячейки идут подряд, как в ноутбуке; вывод под чертой снят запуском.

1 Что такое модель

Объект — то, о чём спрашивают: пингвин, письмо, квартира. Он описан d числами, *признаками*, то есть объект — вектор $x \in \mathbb{R}^d$. У объекта есть *ответ* y. Если ответ — одна из нескольких меток, задача — *классификация*, метки — *классы*, и их нумеруют $0, 1, \ldots, K-1$; если число — *регрессия*.

Пример на все пять уроков — двенадцать пингвинов трёх видов: Адели (класс 0), антарктический (1) и папуанский (2). Числа придуманы, но похожи на настоящие. Признаки — длина клюва в миллиметрах и масса в граммах.

Модель — функция $a(x)$, объекту — ответ. Её строят по примерам с известными ответами: это *обучение*, а примеры — *обучающая выборка*, train. Потом модель *предсказывает* ответы новых объектов. То, что задают до обучения, — число соседей, глубину дерева, — называют *гиперпараметрами*. Хорошая модель верно отвечает на новых объектах, а не на тех, что видела. Поэтому качество меряют на *тестовой выборке*, test: её откладывают заранее и в обучении не используют.

Объекты train — строки таблицы X размера $n \times d$, ответы — вектор y; шесть пингвинов теста — `Xt` и `yt`:

```
import numpy as np
# клюв, мм; масса, г
X = np.array([[38, 3600], [40, 3800], [36, 3300],
              [41, 3950], [47, 3550], [50, 3800],
              [46, 3400], [51, 3950], [46, 4900],
              [49, 5400], [41, 4650], [50, 5600]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2])
Xt = np.array([[41, 3520], [39, 3880], [48, 3700],
               [45, 3450], [47, 5150], [44, 4750]])
yt = np.array([0, 0, 1, 1, 2, 2])
print(X.shape, np.bincount(y))
```

```
(12, 2) [4 4 4]
```

Качество классификации — *доля верных ответов*, accuracy, регрессии — *среднеквадратичная ошибка*, MSE:

$$\text{acc} = \frac{1}{n}\sum_{i=1}^{n}[a(x_i) = y_i], \qquad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\big(a(x_i) - y_i\big)^2,$$

где $[\dots]$ — 1, если условие верно, и 0, если нет. Точка отсчёта, baseline, — модель, которая ничего не учит: константа «всегда класс 0» угадывает на этом тесте треть. Модель, которая её не обходит, бесполезна.

Ловушка. Качество на train ничего не говорит о новых объектах. Модель «увидел знакомый объект — верни его ответ» на train безошибочна, а на новых беспомощна. Чем гибче модель, тем легче ей подогнаться под train вместе с его случайностями; это *переобучение*, и видно оно только на отложенных данных.

2 Ближайшие соседи

Похожие объекты обычно отвечают похоже. Модель k *ближайших соседей* для объекта q находит k ближайших к нему обучающих объектов и отдаёт ответ, которого среди них больше. Обучения нет: модель — сама таблица. Близость меряют расстоянием — *евклидовым*, L2, или *манхэттенским*, L1, путём вдоль осей:

$$\rho_2(x,q) = \sqrt{\sum_{j=1}^{d}(x_j - q_j)^2}, \qquad \rho_1(x,q) = \sum_{j=1}^{d}|x_j - q_j|.$$

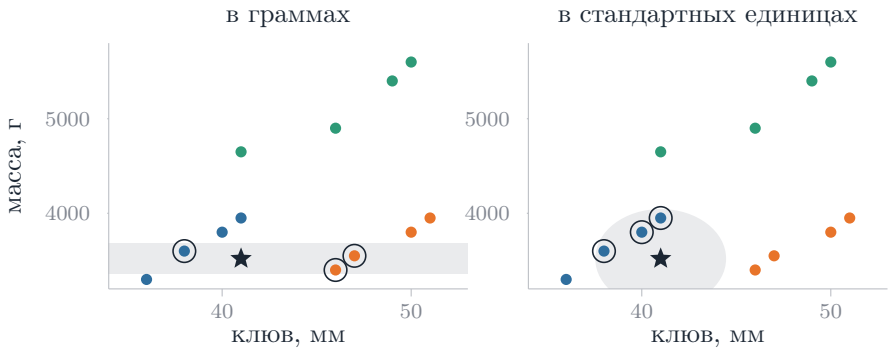
Расстояния от первого пингвина теста до всех двенадцати — через распространение; соседи — первые k номеров по возрастанию расстояния, `argsort`; голосование — счёт классов среди соседей, `bincount`, и номер самого частого, `argmax`:

```
q = Xt[0]
d2 = np.sqrt(((X - q) ** 2).sum(axis=1))
d1 = np.abs(X - q).sum(axis=1)
print(d2.round(1))
print(d1)
nb = np.argsort(d2, kind="stable")[:3]
votes = np.bincount(y[nb], minlength=3)
print(nb, y[nb], votes, votes.argmax())
```

```
[  80.1  280.   220.1  430.    30.6  280.1  120.1
  430.1 1380.  1880.  1130.  2080. ]
[  83  281  225  430   36  289  125  440 1385 1888
 1130 2089]
[4 0 6] [1 0 1] [1 2 0] 1
```

Соседи — пингвины 4, 0 и 6, голоса 1 : 2 : 0, ответ — класс 1. При равных голосах `argmax` берёт меньший номер класса; так решает и sklearn, но это лишь одно из правил, и в задаче правило ничьей задаёт условие.

Ответ — антарктический, а пингвин — Адели. Оба расстояния — почти одна разность масс: масса различается на сотни граммов, клюв — на единицы миллиметров, и клюв, которым Адели и отличается, в расстоянии почти не участвует. На рисунке Адели синие, антарктические оранжевые, папуанские зелёные, вопрос — звезда, соседи обведены. Слева — кого модель считает близким в граммах: всех, чья масса около 3520, при любом клюве.



Масштаб признаков

Стандартизация вычитает из признака среднее μ_j и делит на стандартное отклонение σ_j: $z_j = (x_j - \mu_j)/\sigma_j$, где $\sigma_j^2 = \frac{1}{n}\sum_i (x_{ij} - \mu_j)^2$. После неё у каждого признака среднее 0 и разброс 1, и разница на единицу значит одно и то же по любой оси. *Min-max* переводит признак в отрезок $[0, 1]$: $z_j = (x_j - m_j)/(M_j - m_j)$, где m_j и M_j — наименьшее и наибольшее значения.

Правило. Среднее и разброс — или наименьшее и наибольшее — считают по train и теми же числами преобразуют test. Тест — это будущие объекты, их статистики при обучении неизвестны, а модель, обученная в одних единицах, должна получать новые объекты в тех же.

Соседи для всех шести пингвинов теста, в граммах и в стандартных единицах; цикл — по объектам теста, внутри — те же три строки:

```
mu, sd = X.mean(axis=0), X.std(axis=0)
Z, Zt = (X - mu) / sd, (Xt - mu) / sd
print(mu.round(1), sd.round(1))

def knn(Xtr, ytr, q, k):
    d = np.sqrt(((Xtr - q) ** 2).sum(axis=1))
    nb = np.argsort(d, kind="stable")[:k]
    return np.bincount(ytr[nb]).argmax()

for A, B in [(X, Xt), (Z, Zt)]:
    pred = np.array([knn(A, y, q, 3) for q in B])
    print(pred, (pred == yt).mean())
```

```
[  44.6 4158.3] [  4.9 750. ]
[1 0 0 1 2 2] 0.6666666666666666
[0 0 1 1 2 2] 1.0
```

В граммах модель ошиблась на двух пингвинах из шести, в стандартных единицах — ни на одном. Справа на рисунке выше — окрестность первого пингвина теста после стандартизации: эллипс, а не полоса.

Регрессия и веса

В регрессии голосование заменяют средним: ответ — среднее y у k соседей. Масса по длине крыла в миллиметрах, три соседа для крыла 205 мм:

```
fl = np.array([185, 190, 182, 195, 193, 198,
               189, 201, 214, 221, 210, 224])
m = X[:, 1]
nb = np.argsort(np.abs(fl - 205), kind="stable")[:3]
print(fl[nb], m[nb], m[nb].mean().round(1))
```

```
[201 210 198] [3950 4650 3800] 4133.3
```

Во *взвешенных соседях* ближний сосед весит больше дальнего: голос соседа x_i — вес w_i, убывающий с расстоянием, например $w_i = 1/\rho(x_i, q)$. Класс — тот, у которого больше сумма весов, а в регрессии ответ — взвешенное среднее:

$$a(q) = \arg\max_c \sum_{i \in N_k(q)} w_i \,[y_i = c], \qquad a(q) = \frac{\sum_{i \in N_k(q)} w_i y_i}{\sum_{i \in N_k(q)} w_i},$$

где $N_k(q)$ — номера k ближайших к q. При равных весах это обычное большинство и обычное среднее.

Выбор k и скользящий контроль

При $k = 1$ модель повторяет каждую случайность train: один нетипичный пингвин отвоёвывает себе окрестность. При $k = n$ соседи — все, и модель — константа. Хорошее k между ними, и выбирать его по test нельзя: test тогда станет обучающим, и его оценка перестанет быть честной.

Выбирают *скользящим контролем*, cross-validation. Train делят на m частей; каждую по очереди откладывают, модель учат на остальных и меряют на отложенной; итог — среднее m замеров. Крайний случай $m = n$ — *контроль по одному*, leave-one-out: каждый объект предсказывается по всем остальным, LOO $= \frac{1}{n}\sum_i [a_{-i}(x_i) = y_i]$, где a_{-i} — модель без объекта i. Берут то k, у которого оценка лучше.

Ловушка. Отложенный объект не сосед самому себе. Если искать соседей x_i среди всех объектов, ближайшим окажется он сам, на расстоянии 0, и при $k = 1$ модель «угадает» каждый объект: LOO выйдет 1 почти на любых данных. Это та же проверка на train, только переодетая.

3 То же в sklearn

У каждой модели sklearn один порядок: создать объект с гиперпараметрами, `fit(X, y)` — обучить, `predict(X)` — предсказать. Выученные числа — в полях с подчёркиванием на конце: `coef_`, `cluster_centers_`. У преобразований признаков вместо `predict` — `transform`. Этот порядок одинаков во всех пяти уроках.

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

sc = StandardScaler().fit(X)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(sc.transform(X), y)
print(model.predict(sc.transform(Xt)))
```

```
[0 0 1 1 2 2]
```

Правило. `fit` у преобразования — только на train, к test — `transform` с теми же числами. `fit_transform` на test — та самая ошибка, от которой предостерегает раздел 2.

Один признак — столбцом, `x[:, None]`. L1 — `p=1`, взвешенные соседи — `weights="distance"`, регрессия — `KNeighborsRegressor`.

Ловушка. Ничьи у `KNeighborsClassifier`. При равных голосах — меньший номер класса, как у `argmax`. При равных расстояниях порядок соседей не обещан и зависит от способа поиска, `algorithm`. Сетка 7×7 целых точек, классы 0 и 1 через одну, вопрос — в центре клетки, на равном расстоянии от четырёх её углов:

```
g = np.arange(-3, 4)
P = np.array([(i, j) for i in g for j in g])
lab = np.arange(49) % 2
for alg in ["brute", "kd_tree"]:
    nn = KNeighborsClassifier(1, algorithm=alg)
    nn.fit(P, lab)
    print(alg, nn.predict([[0.5, 0.5]]))
```

```
brute [1]
kd_tree [0]
```

Один сосед, два способа поиска — два ответа. Где правило ничьей важно, а в задаче его задаёт условие, соседей считают сами.

Дальше — урок «Решающие деревья»: модель, которая не считает расстояний, и масштаб признаков ей поэтому не важен.